RAPPORT – Mini Task Manager

Auteurs : DEHMANI Manar – SIARRI Julie
Date : 22 Mars 2026

1. Introduction

Dans le cadre du module **Virtualisation**, l'objectif de ce projet était de concevoir, conteneuriser et déployer une application fullstack selon une architecture microservices sur Kubernetes.

Ce projet vise à mettre en pratique les concepts suivants :

- Architecture microservices
- Containerisation avec Docker
- Orchestration avec Kubernetes
- Isolation via Namespace
- Gestion des permissions avec RBAC
- Déploiement d'une base de données persistante avec StatefulSet
- Gestion des secrets et de la configuration
- Exposition via Ingress

L'application développée est un **Mini Task Manager**, permettant la gestion de tâches avec un système de rôles utilisateurs.

2. Présentation de l'application

2.1 Objectif fonctionnel

L'application permet :

- La création d'utilisateurs
- L'attribution automatique d'un rôle (admin ou user)
- L'ajout, modification et suppression de tâches
- L'affichage des tâches propres à l'utilisateur connecté
- Une vue administrateur permettant de visualiser les utilisateurs

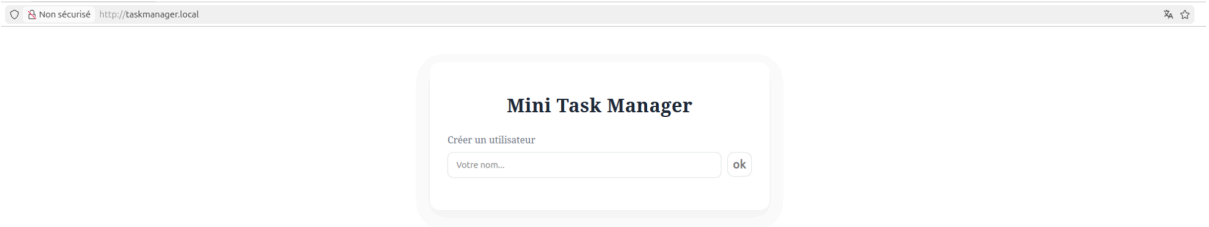
2.2 Interface utilisateur

L'interface utilisateur a été développée en Angular 20 en utilisant des Standalone Components et le système de Signals pour la gestion réactive des données.

L'objectif principal de l'interface est d'être simple, claire et fonctionnelle, tout en permettant la gestion des rôles (admin / user).

2.2.1 Écran de création utilisateur

Au premier lancement de l'application, aucune session n'est active.
L'utilisateur doit donc créer un profil en renseignant son nom.



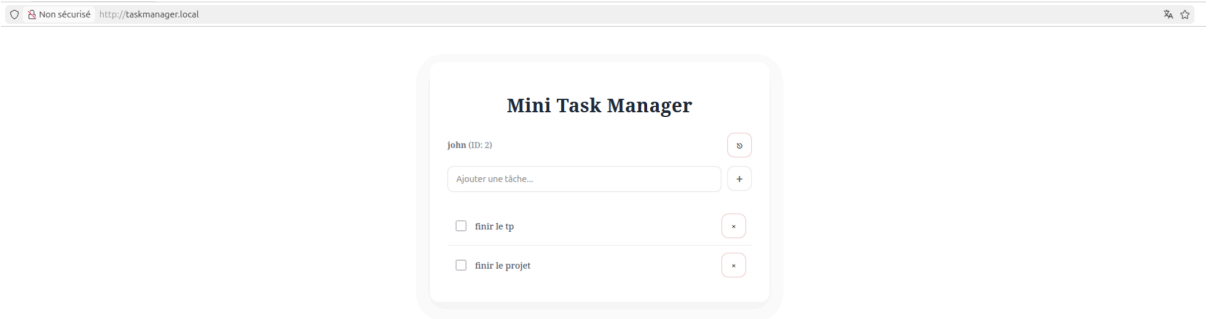
Lors de la création :

- Le premier utilisateur créé devient automatiquement administrateur.
- Les utilisateurs suivants obtiennent le rôle "user".
- Les informations (id, rôle, nom) sont stockées dans le localStorage du navigateur afin de simuler une session.

Ce mécanisme permet de gérer dynamiquement les permissions dans l'application.

2.2.2 Interface principale – Utilisateur standard

Une fois connecté, l'utilisateur accède à l'interface principale.



Cette interface permet :

- L'ajout d'une nouvelle tâche via un champ de saisie
- La validation de la tâche (bouton +)
- La modification de l'état d'une tâche (case à cocher)
- La suppression d'une tâche (bouton ×)

Chaque action déclenche un appel HTTP vers le Task Service.

2.2.3 Gestion dynamique des tâches

Le fonctionnement interne repose sur :

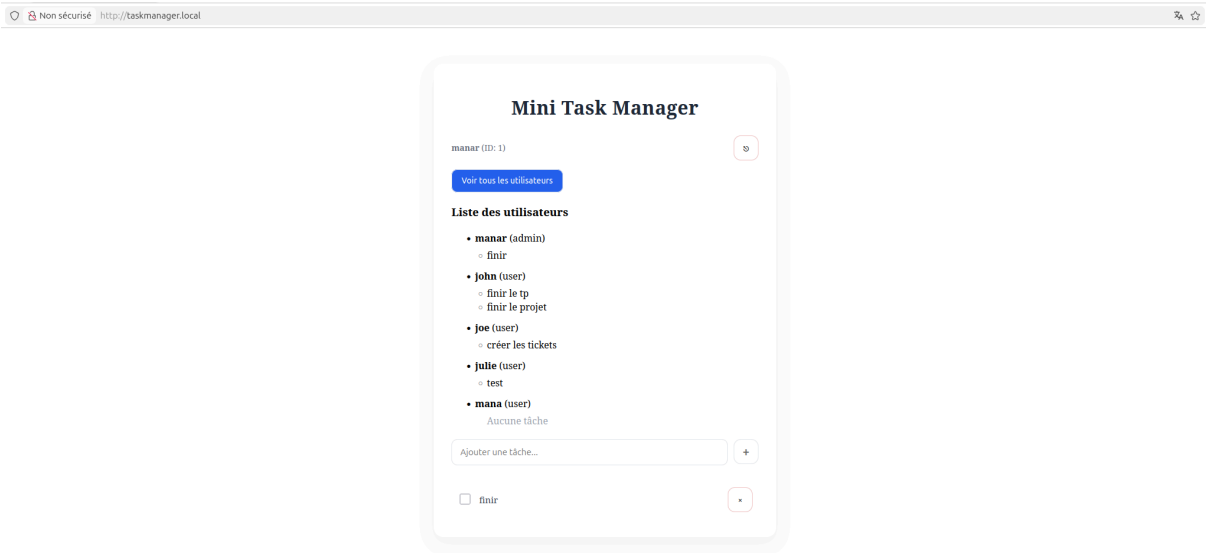
- Des requêtes REST vers le backend
- Une mise à jour réactive via Angular Signals
- Une synchronisation immédiate avec la base MySQL

Lorsqu'une tâche est :

- ajoutée → elle est insérée en base puis affichée
- modifiée → son état est mis à jour
- supprimée → elle est retirée de la base et de l'interface

2.2.4 Interface Administrateur

Si l'utilisateur possède le rôle "admin", des fonctionnalités supplémentaires sont affichées dynamiquement.



L'administrateur peut :

- Visualiser la liste complète des utilisateurs
- Voir les rôles associés
- Consulter les tâches de chaque utilisateur

L'interface adapte son affichage en fonction du rôle récupéré depuis le backend.

Cela démontre :

- Une gestion des permissions côté serveur
- Une adaptation conditionnelle côté frontend

2.2.5 Gestion des rôles

Le système de rôles fonctionne comme suit :

- Lors de la création d'un utilisateur, le backend vérifie si un admin existe.
- Si aucun admin n'est présent, le premier utilisateur devient admin.
- Les autres utilisateurs sont automatiquement affectés au rôle user.

Les rôles déterminent :

- L'accès à certaines routes API
- L'affichage conditionnel de certains composants Angular

2.2.6 Déconnexion

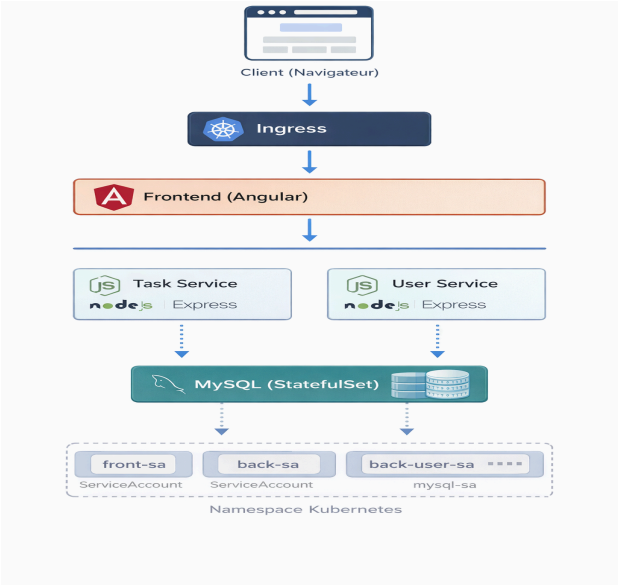
Un bouton de déconnexion permet :

- La suppression des données stockées dans le localStorage
- Le retour à l'écran de création utilisateur
- La réinitialisation de l'état Angular

3. Architecture technique

L'application repose sur une architecture microservices.

3.1 Architecture



3.2 Description des composants

3.2.1 Frontend

- Angular 20
- Standalone Components
- Signals
- HttpClient
- Communication REST avec les microservices

3.2.2 Task Service

- Node.js
- Express
- CRUD des tâches
- Vérification des rôles
- Connexion MySQL

3.2.3 User Service

- Création utilisateur
- Attribution du rôle
- Liste des utilisateurs

3.2.4 Base de données

- MySQL 8
- Déployée via StatefulSet
- PersistentVolumeClaim
- ConfigMap
- Secret

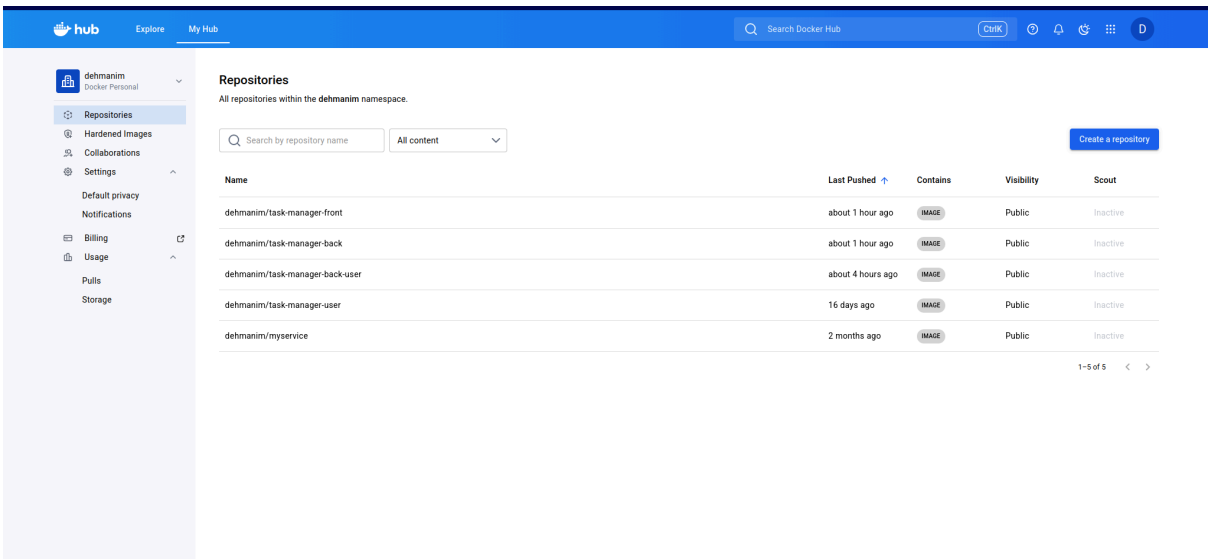
Cette architecture garantit la persistance des données même en cas de redémarrage du pod.

4. Containerisation avec Docker

Chaque composant dispose de son propre Dockerfile.

Les images ont été :

- Construites localement
- Testées
- Publiées sur Docker Hub



5. Déploiement Kubernetes

5.1 Namespace

Un namespace dédié `task-manager` a été créé afin d'isoler toutes les ressources.

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   30d
ingress-nginx     Active   28d
kube-node-lease   Active   30d
kube-public       Active   30d
kube-system       Active   30d
task-manager      Active   4h8m
manar.dehmani@panda:~/projet-virtualisation$
```

5.2 Deployments

Les services sont déployés via :

- front-deployment
- back-deployment
- back-user-deployment
- mysql-statefulset

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get pods -n task-manager
NAME                                  READY   STATUS    RESTARTS   AGE
task-manager-back-7b74d4f869-45556    1/1     Running   0          89m
task-manager-back-user-7db6576568-xc4zh   1/1     Running   0          177m
task-manager-front-8498fc5c88-4w7vw   1/1     Running   0          80m
task-manager-mysql-0                  1/1     Running   0          4h8m
manar.dehmani@panda:~/projet-virtualisation$
```

5.3 Services Kubernetes

- ClusterIP pour les communications internes
- Ingress pour l'exposition externe

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get svc -n task-manager
NAME                            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
task-manager-back-service       ClusterIP   10.100.192.145   <none>        3000/TCP   4h9m
task-manager-back-user-service  ClusterIP   10.109.31.220    <none>        3000/TCP   4h9m
task-manager-front-service      ClusterIP   10.98.56.210     <none>        80/TCP     4h9m
task-manager-mysql              ClusterIP   10.102.73.216    <none>        3306/TCP   4h9m
manar.dehmani@panda:~/projet-virtualisation$
```

5.4 Base de données – StatefulSet

MySQL est déployé via StatefulSet pour :

- Garantir une identité stable
- Assurer la persistance des données

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get pvc -n task-manager
NAME                                STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
SCLASS   AGE
mysql-storage-task-manager-mysql-0  Bound    pvc-ec0d166c-5abd-4f35-89fd-1f39c6fd0971   1Gi        RWO            standard       <unset>
         4h13m
manar.dehmani@panda:~/projet-virtualisation$
```

Un ConfigMap est utilisé pour la configuration.
Un Secret est utilisé pour les identifiants.

6. Gateway (Ingress)

Un gateway a été ajouté via Ingress.

L'accès se fait via : http://taskmanager.local/

Cela permet :

- Un point d'entrée unique
- Un routage vers le frontend
- Une séparation claire des services internes

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get ingress -n task-manager
NAME                   CLASS   HOSTS               ADDRESS        PORTS   AGE
task-manager-ingress   nginx   taskmanager.local   192.168.49.2   80      4h17m
manar.dehmani@panda:~/projet-virtualisation$
```

7. Ajout d'un deuxième service

Le projet comprend deux microservices backend :

- Task Service
- User Service

Ces services communiquent indirectement via la base de données.

L’architecture respecte donc le pattern demandé dans l’énoncé.

8. Base de données

Une base MySQL a été ajoutée :

- StatefulSet
- Volume persistant
- ConfigMap
- Secret

9. Sécurisation du cluster

9.1 RBAC

La sécurité RBAC (Role-Based Access Control) a été configurée afin de contrôler les permissions des microservices au sein du namespace Kubernetes.Au lieu de créer des rôles pour des utilisateurs humains (admin/developer/viewer), les permissions sont directement attribuées aux ServiceAccounts utilisés par les pods de l'application.

ServiceAccounts :

- front-sa
- back-sa
- back-user-sa
- mysql-sa

Rôles :

- admin-role
- developer-role
- viewer-role

RoleBindings :

- back-sa → developer-role
- back-user-sa → developer-role
- front-sa → viewer-role

Cela garantit que :

- Les services backend disposent des permissions nécessaires pour gérer les ressources Kubernetes (deployments, services).

- Le frontend dispose d'un accès en lecture seule.

- La base de données fonctionne avec son propre ServiceAccount isolé.

```
manar.dehmani@panda:~/projet-virtualisation$ kubectl get rolebindings -n task-manager
NAME                ROLE                   AGE
back-binding        Role/developer-role    4h41m
back-user-binding   Role/developer-role    4h41m
front-binding       Role/viewer-role       4h41m
manar.dehmani@panda:~/projet-virtualisation$
```

9.2 Test des permissions

Les permissions ont été vérifiées avec :

```
manar.dehmani@panda:~/projet-virtualisation/task-manager-front$ kubectl auth can-i create deployments \
--as=system:serviceaccount:task-manager:back-sa \
-n task-manager
yes
manar.dehmani@panda:~/projet-virtualisation/task-manager-front$ kubectl auth can-i get pods \
--as=system:serviceaccount:task-manager:front-sa \
-n task-manager
yes
```

Conclusion

En guise de conclusion, il paraît clair que le projet mini Task Manager a permis de mettre en pratique :

- La conception d'une architecture microservices
- La containerisation complète d'une application fullstack
- Le déploiement et l'orchestration via Kubernetes
- La mise en place d'un gateway
- La sécurisation via RBAC
- La gestion de la persistance des données

L'utilisation de Kubernetes permet de structurer, sécuriser et rendre scalable une application distribuée.

Nous avons essayé de faire en sorte que ce projet représente une mise en œuvre complète des concepts étudiés en virtualisation et orchestration et respecte l'ensemble des exigences de l'énoncé et intègre un maximum de technologies demandées.